Monad DB

Security Assessment

March 20th, 2026 — Prepared by OtterSec

James Wang james.wang@osec.io

Jinwoo Lee jin@osec.io

Table of Contents

Executive Summary **2**

Overview 2

Key Findings 2

Scope **3**

Findings **4**

Vulnerabilities **5**

OS-MDB-ADV-00 | Non-Contiguous Node Write Risk 6

OS-MDB-ADV-01 | Incorrect Trie Range Intersection Check 8

OS-MDB-ADV-02 | Implementation-Defined Bit-Field Layout Risk 10

OS-MDB-ADV-03 | Memory Leak due to Missing Finalization for Empty Values 11

General Findings **13**

OS-MDB-SUG-00 | Duplicate Parent Entry in Rewrite Stack 14

Vulnerability Rating Scale **16**

Procedure **17**

© 2026 Otter Audits LLC. All Rights Reserved.

01 — Executive Summary

Overview

Category Labs engaged OtterSec to assess the `db` program. This assessment was conducted between February 9th and February 27th, 2026. For more information on our auditing methodology, refer to chapter 07.

Key Findings

We produced 5 findings throughout this audit engagement.

In particular, we identified a vulnerability where the logic responsible for writing a node may replace the writer after partially serializing a node, and concurrent writes during polling may advance the write cursor, resulting in the remaining bytes to be written at non-contiguous offsets (OS-MDB-ADV-00). Additionally, during path traversal, the prefix check may incorrectly reject prefixes that still contain keys within the requested range, potentially ending the traversal prematurely and resulting in missed matching entries (OS-MDB-ADV-01).

Furthermore, the code assumes a specific bit-field ordering when extracting the count value, but C++ bit-field layout is implementation-defined and may differ across compilers, resulting in incorrect decoding of offsets if the compiler utilizes a different packing or bit ordering (OS-MDB-ADV-02).

We also advised to avoid pushing the final (parent, child-index) pair in copy-trie implementation, when it matches the stack's current top entry to prevent redundant node serialization and metadata updates during the upward rewrite phase (OS-MDB-SUG-00).

© 2026 Otter Audits LLC. All Rights Reserved.

02 — Scope

The source code was delivered to us in a git repository at https://github.com/category-labs/monad. This audit was performed against commit 318fcd3.

A brief description of the programs is as follows:

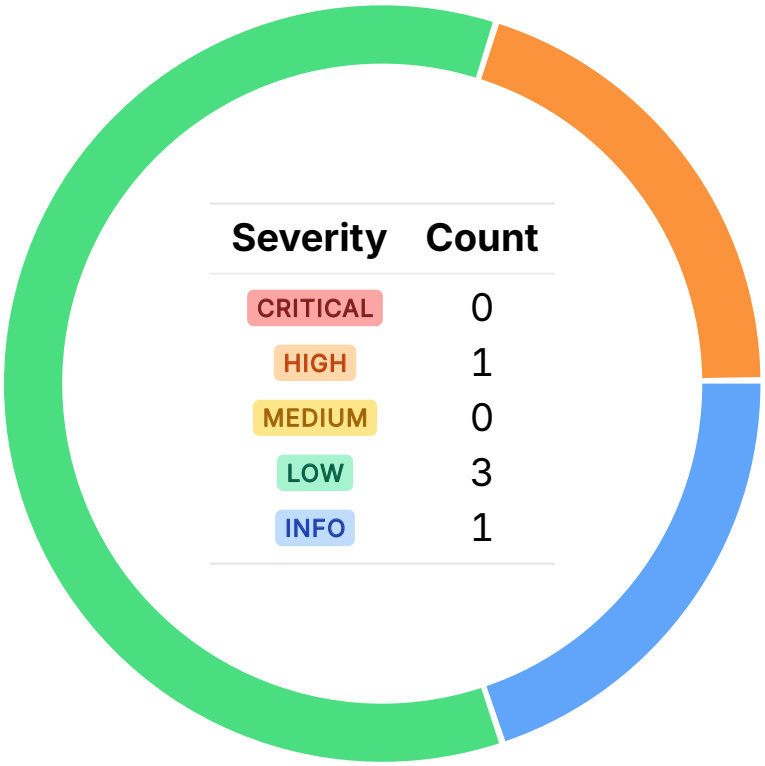
Name	Description
db	It implements the storage and access layer for the Merkle Patricia Trie, supporting versioned lookups, updates, traversal, and subtree operations. It also manages asynchronous disk I/O, caching, and worker-thread execution for efficient on-disk trie access.

© 2026 Otter Audits LLC. All Rights Reserved.

03 — Findings

Overall, we reported 5 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



Severity	Count
CRITICAL	0
HIGH	1
MEDIUM	0
LOW	3
INFO	1

© 2026 Otter Audits LLC. All Rights Reserved.

04 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities we identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in chapter 06.

ID	Severity	Status	Description
OS-MDB-ADV-00	HIGH	RESOLVED	`async_write_node` may replace the writer after partially serializing a node, and concurrent writes during polling may advance the write cursor, resulting in the remaining bytes to be written at non-contiguous offsets.
OS-MDB-ADV-01	LOW	RESOLVED	The prefix check `min_.starts_with(path)` may incorrectly reject prefixes that still contain keys within the requested [`min`, `max`) range. As a result, valid trie subtrees may be pruned early, resulting in the range traversal to miss matching entries.
OS-MDB-ADV-02	LOW	RESOLVED	The code assumes a specific bit-field ordering when extracting the count value, but C++ bit-field layout is implementation-defined and may differ across compilers, resulting in incorrect decoding of offsets if the compiler utilizes a different packing or bit ordering.
OS-MDB-ADV-03	LOW	RESOLVED	The Rust `read()` wrapper returns early for zero-length values without calling `triedb_finalize()`, even though the C++ `triedb_read()` may still allocate a buffer.

© 2026 Otter Audits LLC. All Rights Reserved.

Non-Contiguous Node Write Risk HIGH OS-MDB-ADV-00

Description

In `trie`, the `while` loop in `async_write_node` writes a node incrementally across buffers and replaces the writer when the current buffer is exhausted. However, `async_write_node` assumes the continuation writer returned by `replace_node_writer` will remain the correct immediate successor for the current node's remaining bytes. That assumption is fragile because `replace_node_writer` may internally poll or allow other writes to progress, potentially advancing the global write cursor beyond the expected contiguous location.

>_ *category/mpt/trie.cpp* cpp

```cpp
// return physical offset the node is written at
async_write_node_result async_write_node(
    UpdateAuxImpl &aux, node_writer_unique_ptr_type &node_writer,
    Node const &node)
{
retry:
    [...]
    else {
        [...]
        else {
            [...]
            if (!new_node_writer) {
                goto retry;
            }[...]
        }
        [...]
        while (offset_in_on_disk_node < size) {
            [...]
            if (offset_in_on_disk_node < size &&
                node_writer->sender().remaining_buffer_bytes() == 0) {
                // replace node writer
                new_node_writer = replace_node_writer(aux, node_writer);
                if (new_node_writer) {
                    // initiate current node writer
                    MONAD_DEBUG_ASSERT(
                        node_writer->sender().written_buffer_bytes() ==
                        node_writer->sender().buffer().size());
                    node_writer->initiate();
                    // shall be recycled by the i/o receiver
                    node_writer.release();
                    node_writer = std::move(new_node_writer);
                }
            }
        }
    }
    return ret;
}
```

© 2026 Otter Audits LLC. All Rights Reserved.

If this occurs, the remaining bytes of the current node may be written at a non-adjacent offset even though the node's starting offset assumes contiguous storage. Additionally, the retry logic (`goto retry`) cannot rollback any bytes written before the poll. While local variables are reset, the partially written prefix remains in the old writer, effectively wasting buffer space.

Remediation

Acquire the next consecutive writer before committing any bytes that require continuation, ensuring the full node may be written contiguously without relying on post-write writer replacement.

Patch

Resolved in PR#2082.

© 2026 Otter Audits LLC. All Rights Reserved.

Incorrect Trie Range Intersection Check LOW OS-MDB-ADV-01

Description

In `traverse_util`, the current prefix check is asymmetric, when `path` is shorter than `min_`, it only tests whether `min_` starts with path. That is sufficient only for prefixes that lie on the lower-bound frontier itself, but it is not a complete test for range intersection. A trie prefix may fail to be a prefix of `min_` and still represent keys that fall inside [`min_`, `max_`). For example, in cases like `(min, max) = (0x01234, 0x1234)` and `path = 0x1`, the check returns false even though keys beginning with `0x1` may still satisfy `min <= key < max`. As a result, the traversal logic may incorrectly prune valid subtrees.

>_ *category/mpt/traverse_util.hpp* cpp

```cpp
class RangedGetMachine : public TraverseMachine
{
    Nibbles path_;
    Nibbles const min_;
    Nibbles const max_;
    TraverseCallback callback_;

private:
    // This function is a looser version checking if min <= path < max. But it
    // will also return true if we should continue traversing down. Suppose we
    // have the range [0x00, 0x10] and we are at node 0x0. In that case the
    // path's size is less than the min, check if it's as substring.
    bool does_key_intersect_with_range(NibblesView const path)
    {
        bool const min_check = [this, path] {
            if (path.nibble_size() < min_.nibble_size()) {
                return NibblesView{min_}.starts_with(path);
            }
            else {
                return (path >= min_);
            }
        }();
        return min_check && (path < NibblesView{max_});
    }
    [...]
}
```

Remediation

Compare `path` with `min_` if `path` has a shorter length.

© 2026 Otter Audits LLC. All Rights Reserved.

Patch

Resolved in 29ceafa.

© 2026 Otter Audits LLC. All Rights Reserved.

Implementation-Defined Bit-Field Layout Risk LOW OS-MDB-ADV-02

Description

The current implementation in `utils` relies on extracting the most significant 20 bits from the internal representation (`v_`) to obtain the count field. However, the original value is derived from a packed structure (`virtual_chunk_offset_t`) that uses C++ bit fields, whose memory layout and bit ordering are implementation-defined. Different compilers may arrange bit fields differently. Because of this, the assumption that the `count` field consistently occupies the most significant bits of the encoded value may not hold across compilers.

>_ *category/mpt/util.hpp* cpp

```cpp
//! Low resolution offset type that truncates the last 16 bits of
//! virtual_chunk_offset_t, for which we save space for `Node` without
//! losing too much granularity in compaction offset.
class compact_virtual_chunk_offset_t
{
    [...]
public:
    [...]
    constexpr uint32_t get_count() const
    {
        // most significant 20 bits
        static constexpr unsigned count_bits = 20;
        return v_ >> (most_significant_bits - count_bits);
    }
    [...]
};
```

Remediation

Avoid relying on implicit bit-field layout.

Patch

Resolved in c0ca004.

© 2026 Otter Audits LLC. All Rights Reserved.

Memory Leak due to Missing Finalization for Empty Values LOW OS-MDB-ADV-03

Description

The ownership contract between the C++ and Rust layers require Rust to always release the buffer returned by `triedb_read()` utilizing `triedb_finalize()`. In `triedb_read()`, the native side unconditionally performs `*value = new uint8_t[value_len]`, even when `value_len == 0`.

>_ monad-triedb/triedb-driver/src/triedb.cpp cpp

```cpp
int triedb_read(
    triedb *db, bytes key, uint8_t key_len_nibbles, bytes *value,
    uint64_t block_id)
{
    [...]
    auto const &value_view = result.value().node->value();
    if ((value_view.size() >> std::numeric_limits<int>::digits) != 0) {
        // value length doesn't fit in return type
        return -2;
    }
    int const value_len = (int)value_view.size();
    *value = new uint8_t[value_len];
    memcpy((void *)*value, value_view.data(), value_len);
    return value_len;
}
```

However, in the Rust wrapper, `read()` treats `result == 0` as a special-cases and immediately returns `Some(Vec::new())` without calling `triedb_finalize(value_ptr)`. If `triedb_read()` returned a non-null allocation for the empty value case, that allocation is leaked.

>_ monad-triedb/src/lib.rs RUST

```rust
pub fn read(&self, key: &[u8], key_len_nibbles: u8, block_id: u64) -> Option<Vec<u8>> {
    [...]
    if result == 0 {
        return Some(Vec::new());
    }
    // check that there's no unexpected error
    if result <= 0 {
        error!("Unexpected result from triedb_read: {}", result);
        return None;
    }
    let value_len = result.try_into().unwrap();
    let value = unsafe {
        let value = std::slice::from_raw_parts(value_ptr, value_len).to_vec();
        bindings::triedb_finalize(value_ptr);
        value
```

© 2026 Otter Audits LLC. All Rights Reserved.

```
    };
    Some(value)
}
```

Remediation

Ensure `triedb_finalize(value_ptr)` is called even when the returned value length is zero.

Patch

Resolved in 604bcfc.

© 2026 Otter Audits LLC. All Rights Reserved.

05 — General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-MDB-SUG-00	`copy_trie_impl()` may push the same `(parent, child_index)` pair twice onto the stack when inserting the copied subtree, resulting in redundant node serialization and metadata updates during the upward rewrite phase.

© 2026 Otter Audits LLC. All Rights Reserved.

Duplicate Parent Entry in Rewrite Stack OS-MDB-SUG-00

Description

In `copy_trie`, during traversal, `copy_trie_impl()` already pushes `(parent, index)` into `parents_and_indexes` when descending into an existing child. Later, after inserting the new subtree, it unconditionally pushes `(parent, child_index)` again before the upward rewrite phase. This rfesults in the same node to be rewritten and its metadata updated twice during the upward serialization loop.

>_ *category/mpt/copy_trie.cpp* RUST

```
Node::SharedPtr copy_trie_impl(
    UpdateAuxImpl &aux, Node::SharedPtr src_root, NibblesView const src_prefix,
    Node::SharedPtr dest_root, NibblesView const dest_prefix,
    uint64_t const dest_version)
{
    [...]
    while (prefix_index < dest_prefix.nibble_size()) {
        [...]
        // end of node path
        if (node->mask & (1u << nibble)) {
            auto const index = node->to_child_index(nibble);
            if (node->next(index) == nullptr) {
                auto next_node_ondisk =
                    read_node_blocking(aux, node->fnext(index), dest_version);
                MONAD_ASSERT(next_node_ondisk != nullptr);
                node->set_next(index, std::move(next_node_ondisk));
            }
            // there is a matched branch, go to next child
            parent = node.get();
            branch = nibble;
            parents_and_indexes.emplace(std::make_pair(parent, index));
            node = node->next(index);
            node_prefix_index = 0;
            ++prefix_index;
            continue;
        }
        [...]
    }
    [...]
    else {
        MONAD_ASSERT(parent != nullptr);
        auto const child_index = parent->to_child_index(branch);
        // reset child at `branch` to the new_node
        parent->set_next(child_index, std::move(new_node));
        parents_and_indexes.emplace(std::make_pair(parent, child_index));
        [...]
    }

    return dest_root;
}
```

© 2026 Otter Audits LLC. All Rights Reserved.

Remediation

Avoid pushing the final `(parent, child_index)` when it matches the stack's current top entry.

© 2026 Otter Audits LLC. All Rights Reserved.

06 — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the General Findings.

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
- Improperly designed economic incentives leading to loss of funds.

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
- Exploitation involving high capital requirement with respect to payout.

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
- Forced exceptions in the normal user flow.

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
- Improved input validation.

© 2026 Otter Audits LLC. All Rights Reserved.

07 — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.

© 2026 Otter Audits LLC. All Rights Reserved.